

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(Accredited by NAACA+ Approved by AICTE New Delhi & Permanently Affiliated to JNTUH)
Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB PROJECT REPORT



Roll No	:	24911A1228
Name	:	J Govardhan
Project Title	:	Quiz Competition Application
Course Name	:	Object Oriented Programming through Java Lab
Course Code	:	A224592
Academic Year	:	2025-26
Regulations	:	R22
Year and Semester	:	II B.Tech II Semester



CERTIFICATE

This is to certify that the Object Oriented Programming through Java Lab project report entitled “ **Quiz Competition Application** ” submitted by **J Govardhan (24911A1228)** to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech. II year, II Semester in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Faculty

Mr. B Eswar Babu

Associate Professor

Signature of Head of the Department

Dr. A. Obulesu

Professor

Abstract

The Quiz Competition Application is a Java-based evaluation system developed to enable users to participate in timed programming assessments through a graphical user interface. Traditional assessment methods suffer from grading delays and human evaluation errors; therefore, this project aims to provide a user-friendly platform for seamless, automated testing. The application supports dynamic question rendering, a real-time countdown timer, and live leaderboard updates to improve engagement and transparency for users.

The project is designed using core Java concepts such as GUI components, multithreading, and the Java Collections Framework. Threads are used to handle background countdowns independently without freezing the interface, while collections like ArrayList and HashMap are utilized to manage quiz data and store player records efficiently.

The main objective of this project is to create a fast and interactive examination environment where users can test their skills and view results instantly. The system provides an easy-to-use interface that allows users to answer questions and track rankings without delay. The expected outcome of the project is a reliable quiz system that demonstrates real-time performance evaluation using object-oriented programming concepts and Java technologies.

Table of Contents

Chapter No	Title	Page No
1	Introduction	4
2	System Analysis	5
3	System Design	6
4	Implementation	8
5	Testing	11
6	Results	13
7	Conclusion	14

Chapter 1 – Introduction

1.1 Project Overview

The Quiz Competition Application is a multi-threaded Java Swing desktop utility built on core OOP principles. Designed for administering programming MCQs, it separates the background countdown timer from the main Event Dispatch Thread (EDT) to eliminate UI freezing and guarantee real-time responsiveness. The Java Collections Framework (ArrayList and HashMap) handles question storage and dynamically updates a live leaderboard. Real-World use cases include:

- Education: Lightweight, offline tool for lab exams and pop quizzes.
- Corporate Training: Rapid proficiency diagnostic for developer bootcamps.
- Self-Assessment: Local, offline workspace for personalized coding practice.

1.2 Objectives

- Automate Evaluations: Deliver instant scoring and real-time leaderboard updates.
- Eliminate Grading Errors: Automate accuracy by validating user inputs directly against system answer keys.
- Optimize GUI Fluidity: Isolate the timer on a concurrent background thread (Runnable) to prevent interface lags.

1.3 Scope & Scalability

Target Users

- Students: Take timed quizzes, view instant feedback, and track performance.
- Educators: Run automated local assessments without manual grading overhead.

Future Scalability

- Database Integration: Migrate from in-memory HashMap storage to a persistent relational database (MySQL/Oracle) via JDBC.
- Multimedia Support: Expand UI capabilities to include image- and audio-based questions.

Chapter 2 – System Analysis

2.1 Functional Requirements

- Engine Initialization & UI Locking: All option buttons are disabled on launch and unlock only when "Start Quiz" is clicked.
- Dynamic Question Rendering: Sequentially retrieves question objects from an ArrayList to update UI labels and options.
- Independent Background Timer: Runs a dedicated thread for a 10-second countdown per question, updating the GUI safely via `SwingUtilities.invokeLater()`.
- Answer Evaluation & Scoring: Captures selections, immediately stops the timer, validates the answer, and awards 10 points if correct.
- Timeout Management: If the timer hits 0, the question is marked unanswered, scoring is skipped, and the system auto-advances.
- Leaderboard Registration: Prompts for the user's name post-quiz, saves data to an in-memory HashMap, sorts scores in descending order, and updates the live sidebar.

2.2 Non-Functional Requirements

- Performance & Accuracy: Sub-16ms latency for UI updates; timer thread sleeps precisely 1000ms per tick for exact countdowns.
- Reliability & Thread Safety: Background threads handle `InterruptedException` and terminate cleanly on answer selection to avoid leaks.
- Robustness & Validation: Trims whitespaces and blocks empty username inputs to protect scoreboard integrity.
- Encapsulation: Internal game states (scores, keys, indexes) are kept private to ensure OOP security compliance.

2.3 Software Requirements

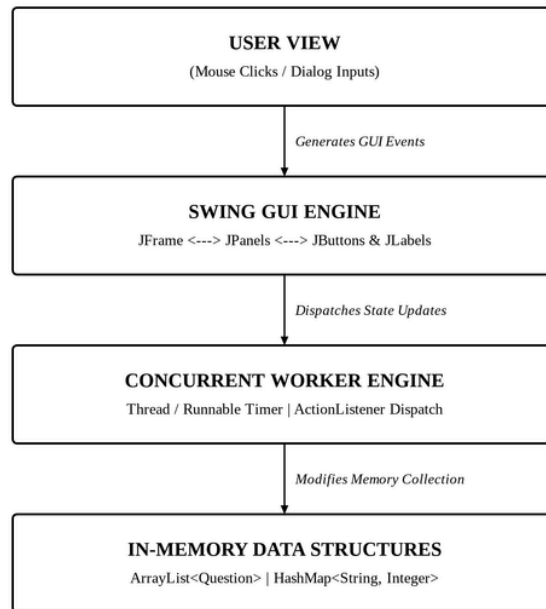
- Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS
- Language & JDK: Java SE, Oracle JDK 8 or above (JDK 11, 17, or 21 recommended)
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or VS Code
- Database System: Transient In-Memory Collections (`java.util.HashMap`)

2.4 Hardware Requirements

- Processor (CPU): Intel Core i3 / AMD Ryzen 3 or higher
- Memory (RAM): 4 GB RAM or above
- Storage: 500 GB HDD or SSD
- Display & Inputs: 1024x768 minimum resolution; standard keyboard and mouse

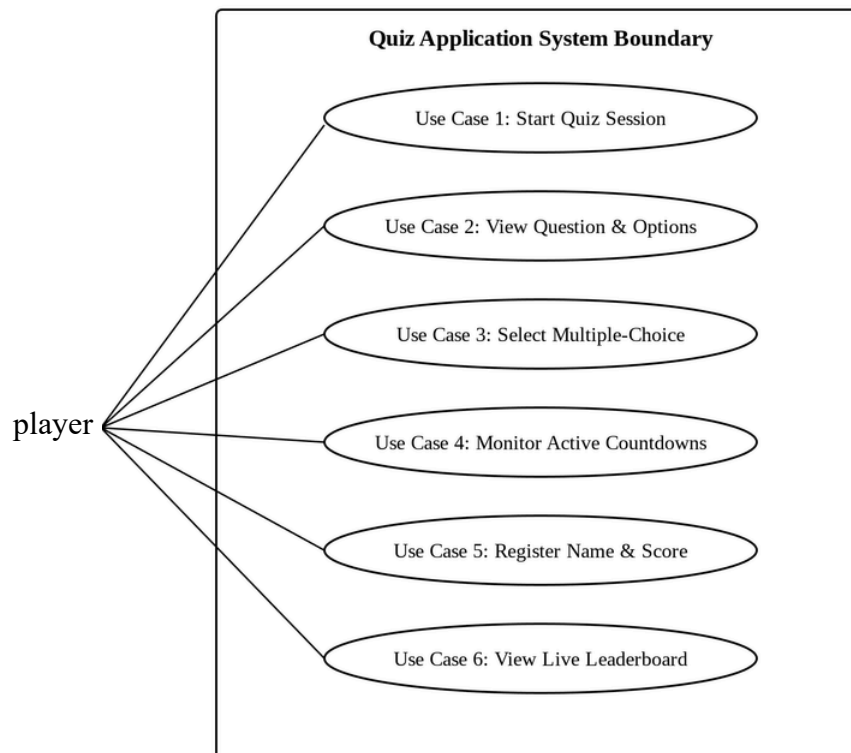
Chapter 3 – System Design

3.1 Architecture Diagram

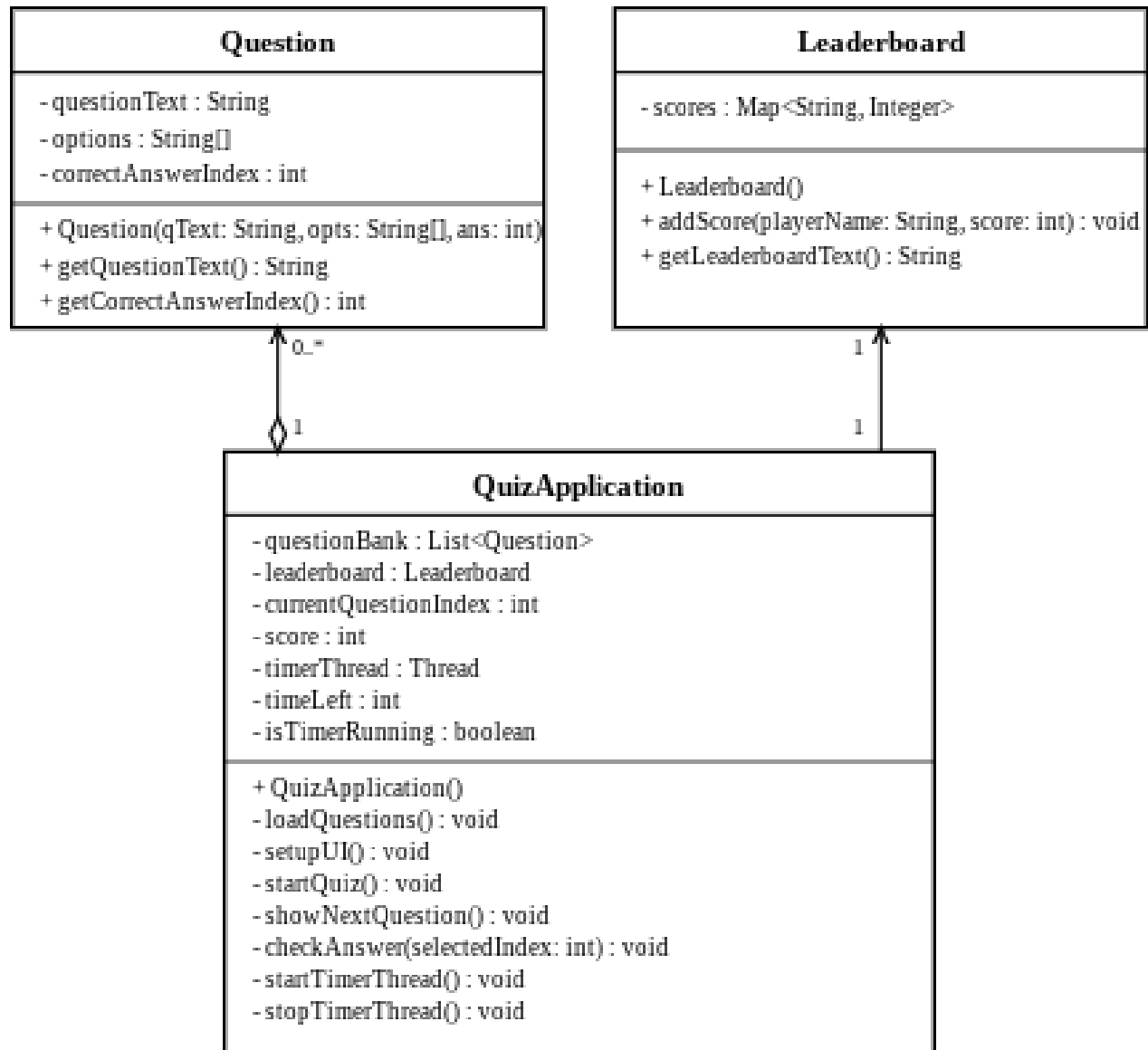


3.2 UML Diagrams

- Use Case Diagram:



- **Class Diagram:**



Chapter 4 – Implementation

4.1 Modules Description

The implementation of the Quiz Competition Application is structured into five cohesive modules that collaborate to handle UI components, background execution threads, and data states:

Module	Description
Question Model Module	Handles structural encapsulation of individual MCQ assets. It models the core attributes: query text, options array (String[]), and the correct answer index key.
Leaderboard Module	Manages player records at runtime. Uses a HashMap collection key-value scheme, converts mapping datasets to lists, and dynamically sorts high scores in descending order.
GUI Layout & Setup Module	Establishes the application's physical structure, utilizing custom nested layout configurations (BorderLayout and GridLayout) to arrange display frames and panels.
Concurrent Timer Module	Handles timer execution on an isolated thread. It tracks a 10-second tick countdown and safe-routes graphical rendering calls back to the primary Event Dispatch Thread (EDT).
Quiz Logic Worker Module	Coordinates overall gameplay flows. It registers mouse inputs, updates active score values, triggers transitions, and prompts user-dialog inputs upon termination.

4.2 Core Java Concepts Applied

The application implements fundamental concepts from the Java syllabus to achieve concurrency, responsiveness, and robust object management:

1. Object-Oriented Principles

- **Classes & Objects:** Divided into dedicated classes (Question, Leaderboard, and QuizApplication) to enforce high cohesion and clean separation of concerns.
- **Inheritance:** The main controller extends `javax.swing.JFrame` (public class QuizApplication extends JFrame), inheriting native window behaviors, layout managers, and structural panels.

2. Java Collections Framework

- **List & ArrayList:** Stores Question objects sequentially in a question bank, allowing fast, index-based access during question transitions.
- **Map & HashMap:** Pairs player names (String) with scores (Integer) as key-value records, which are dynamically sorted and rendered onto the live scoreboard.

3. Concurrency & Thread Safety

- **Multithreading:** Spawns a dedicated background Thread to handle the countdown clock. Running `Thread.sleep(1000)` on this worker thread prevents the main Event Dispatch Thread (EDT) from freezing.
- **Thread-Safe GUI Updates:** Uses `SwingUtilities.invokeLater()` to marshal timer updates from the background thread back onto the EDT, preventing UI rendering collisions.
- **Exception Handling:** Wraps the timer loop in a try-catch block for `InterruptedException`. This ensures the background thread terminates cleanly the moment a user submits an answer or exits.

4.3 Code Snippets

```
private void startTimerThread() {
    stopTimerThread(); // Reset existing execution context safely
    timeLeft = 10;
    isTimerRunning = true;

    timerThread = new Thread() -> {
        while (timeLeft >= 0 && isTimerRunning) {
            SwingUtilities.invokeLater() -> lblTimer.setText("Time Left: " + timeLeft + "s"));
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                return;
            }
            timeLeft--;
        }
        if (isTimerRunning) {
            currentQuestionIndex++;
            showNextQuestion();
        }
    });
    timerThread.start();
}

public String getLeaderboardText() {
    StringBuilder sb = new StringBuilder("--- LEADERBOARD ---\n");
    List<Map.Entry<String, Integer>> list = new ArrayList<>(scores.entrySet());

    list.sort((a, b) -> b.getValue().compareTo(a.getValue()));

    int rank = 1;
    for (Map.Entry<String, Integer> entry : list) {
        sb.append(rank++).append(". ").append(entry.getKey())
        .append(" : ").append(entry.getValue()).append(" pts\n");
    }
    return sb.toString();
}
```

Chapter 5 – Testing

5.1 Test Cases

Test Case ID	Input / Action	Expected Output	Result
TC01	Launch application.	Window renders; start button active; options locked.	Pass
TC02	Click "Start Quiz".	First question loads, options unlock, 10s timer begins.	Pass
TC03	Select any answer.	Timer thread stops, score updates, next question loads.	Pass
TC04	Let timer reach 0s.	Scoring skipped, options lock, next question auto-loads.	Pass
TC05	Complete all questions & enter name.	JOptionPane captures name, saves to HashMap, updates leaderboard.	Pass

5.2 Error Handling

Three core validation and recovery mechanisms guarantee application stability:

1. Concurrency Interruption Handling

To prevent thread collisions or duplicate countdown clocks, the timer loop catches interrupts and exits cleanly if a user submits an answer early:

```
try {  
    Thread.sleep(1000);  
} catch (InterruptedException e) {  
    return;  
}
```

2. Thread-Safe GUI Dispatching

To prevent graphical layout corruption, all timer updates from the background thread are safely delegated to the Event Dispatch Thread (EDT):

```
SwingUtilities.invokeLater(() -> lblTimer.setText("Time Left: " + timeLeft + "s"));
```

3. Input Validation

To protect leaderboard integrity from blank names or whitespace errors, inputs are validated before writing to memory:

```
if (name != null && !name.trim().isEmpty()) {  
    leaderboard.addScore(name.trim(), score);  
    updateLeaderboardDisplay();  
}
```

Chapter 6 – Results and Screenshots

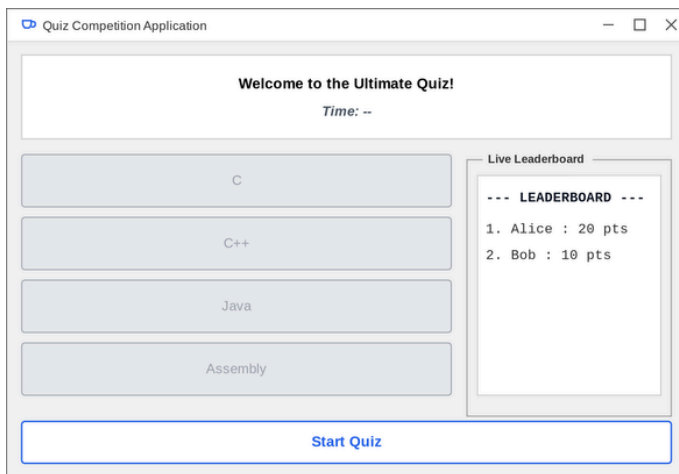


Figure 6.1: Welcome Interface

Default startup interface with a centered window, welcome message, locked options, and active start button.

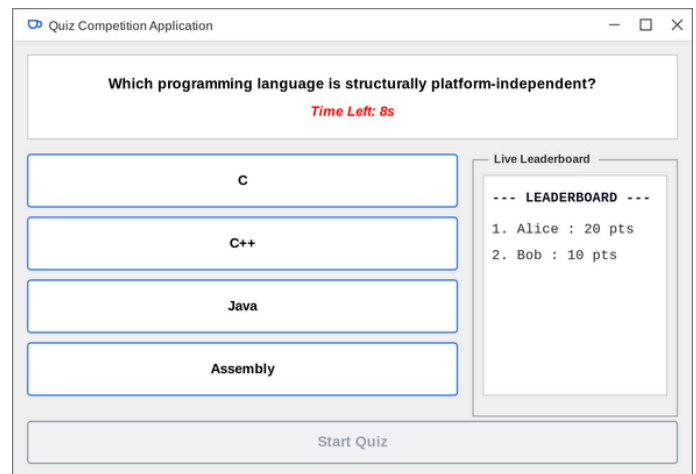


Figure 6.2: Active Gameplay Screen

Active quiz screen showing dynamic question rendering, unlocked options, and background timer ticking down.

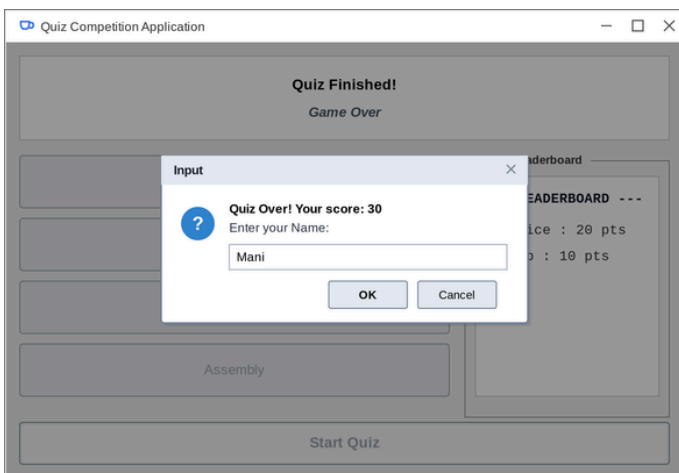


Figure 6.3: Name Input Dialog

Modal dialog prompt overlay capturing player's name and showing final score upon quiz completion.

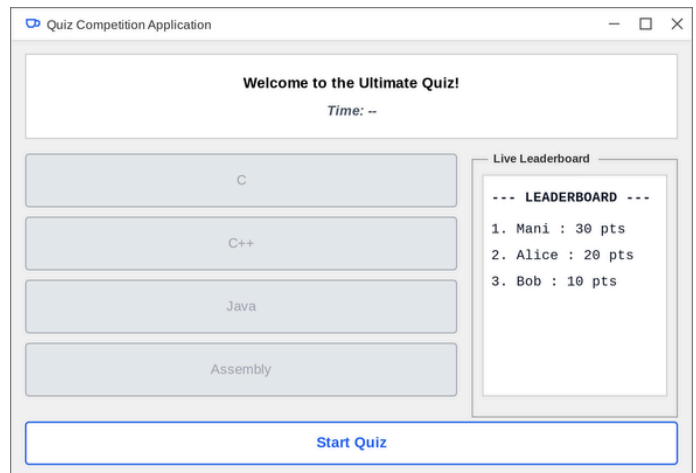


Figure 6.4: Updated Scoreboard

Updated application frame showing new player records added, sorted, and displayed dynamically on the leaderboard.

Chapter 7 – Conclusion and Future Scope

7.1 Conclusion

The Quiz Competition Application has been successfully developed as a robust, interactive, and automated desktop evaluation platform.

Key Achievements

- **Concurrent Execution:** Isolated the 10-second countdown on a background thread, completely eliminating interface freezes.
- **Automated Scoring:** Replaced manual grading with real-time answer validation to guarantee zero evaluation errors.
- **Structured Data Organization:** Utilized the Java Collections Framework to achieve seamless data tracking and instantaneous live rankings.

Learning Outcomes

- Mastered thread lifecycles, nested UI layouts, and safe cross-thread GUI communication via `SwingUtilities.invokeLater()`.
- Gained experience with lambda-based collection sorting for dynamic data management.

7.2 Future Scope

- **Relational Database Integration:** Migrate from in-memory collections to persistent storage (MySQL/Oracle) via JDBC for secure, long-term record keeping.
- **LAN Multiplayer Support:** Implement a client-server architecture using the Java Sockets API to enable live, synchronized classroom competitions.
- **Multimedia Options:** Expand the Java Swing UI components to support image- and audio-based quiz questions.

References

1. Schildt, H. (2018). Java: The Complete Reference, Eleventh Edition. McGraw-Hill Education.
2. Eckel, B. (2006). Thinking in Java, Fourth Edition. Prentice Hall.
3. Oracle Corporation. Java Standard Edition (Java SE) Development Documentation (Core Swing Framework, Event Delegation API & Concurrency Libraries).
4. GitHub Project Repository Reference: <https://github.com/JGovardhan2007/PixelArt-AWT>